

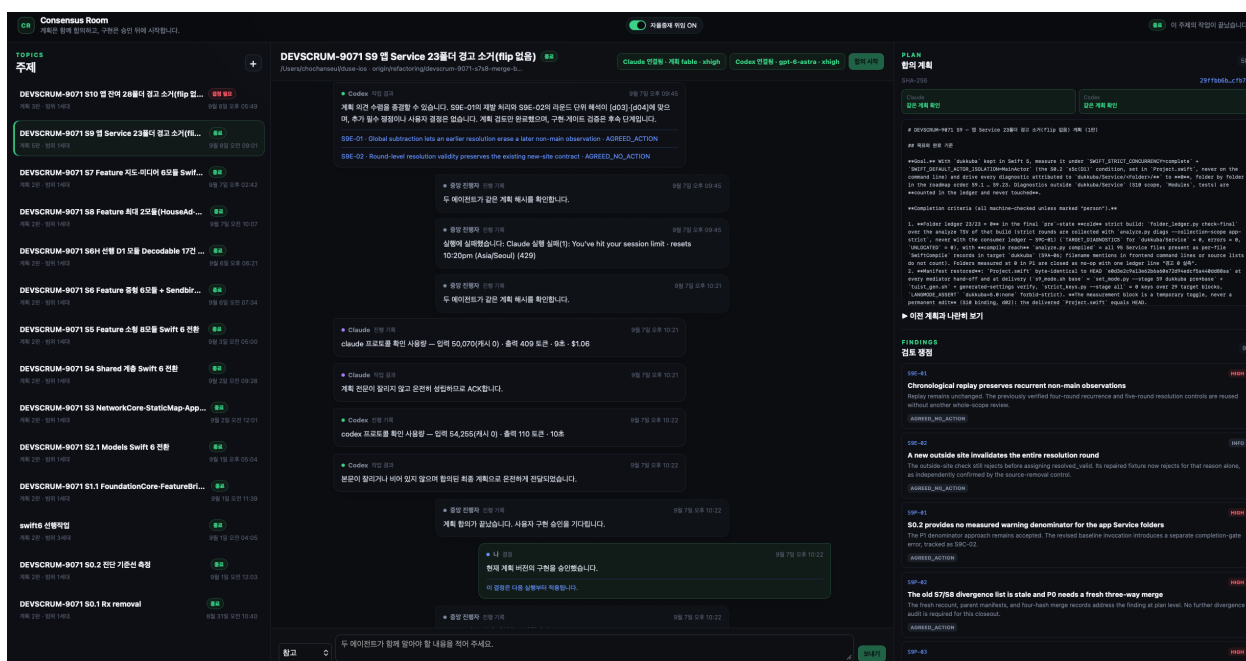
# AI 개발 작업의 계획·구현·검토를 이어 가는 도구

Consensus Room · 설계와 운영 사례 · 조찬슬 · 2026-09-27

## 사람이 반복하던 전달과 확인을 작업 기록으로 옮겼다

Claude와 Codex를 함께 쓰면서 같은 요구사항을 다시 설명하고, 한 도구의 계획을 다른 도구에 복사하고, 수정된 코드를 다시 검토시키는 일이 반복됐다. 대화가 길어질수록 어떤 계획이 승인됐는지, 리뷰가 어느 코드에 대한 것인지 확인하기도 어려워졌다.

이 도구에서 내가 설계한 핵심은 역할을 나누는 것보다 그 사이의 연결을 남기는 데 있다. 계획의 내용, 읽은 원문, 검토한 코드, 실제 커밋을 서로 연결해 다음 행동의 근거로 삼는다. AI가 구현을 수행하더라도 제품 결정과 전달 범위는 별도로 확인한다.



작업 목록과 계획 승인 상태를 함께 보여 주는 기존 운영 화면. 화면 속 모델 배정은 당시 사례다.

## 이번 개편에서 바꾼 중심 흐름

전체 목표는 먼저 정하되 지금 할 단계만 상세히 계획한다. 원문은 위키와 자료 목록에서 필요할 때 나눠 읽는다. 역할에 배정한 AI를 바꿀 수 있게 하고, 계획과 구현은 호환되는 세션에서 이어 간다. 완료 판단은 모델의 성공 문구가 아니라 검증된 코드와 커밋으로 한다.

# 큰 계획을 한 번에 완성하려던 방식의 한계

## 입력을 줄여도 작업 방식이 그대로면 다시 막힌다

초기 흐름은 계획 단계에서 많은 요구사항과 코드를 한꺼번에 모았다. 입력이 커질 때 요약을 더 압축하거나 전달 상한을 높이는 것만으로는 해결되지 않았다. 어떤 근거가 빠졌는지 확인하기 어려웠고, 구현 시점에야 필요한 디자인 정보까지 계획에 들어갔다.

개편에서는 전체 계획과 현재 단계의 계획을 분리했다. 전체 계획은 목표·공통 계약·단계 사이의 의존 관계를 설명한다. 세부 구현은 앞 단계의 실제 결과를 확인한 뒤 구체화한다. 이미 완료된 작업의 판단을 다음 단계마다 다시 만들지 않는 것이 목적이다.

| 상황                | 선택한 처리                    |
|-------------------|---------------------------|
| 원문이 한 호출에 들어가지 않음 | 위치와 버전을 보존하며 여러 번에 나눠 읽는다 |
| 아직 판단할 수 없는 미래 단계 | 개요로 두고 필요한 시점에 상세화한다      |
| 디자인의 세부 수치가 필요함   | 구현자가 해당 화면을 만들 때 확인한다     |
| 계획의 일부 전제가 바뀜     | 영향을 받는 단계만 재계획한다          |
| 이미 검토한 코드가 그대로임   | 유효한 근거와 결과를 이어 사용한다       |

## 상한은 전달을 나누는 기준이다

한 번에 전달할 수 있는 크기와 전체 작업의 예산은 다른 문제다. 전자는 자료를 나눠 읽게 하는 장치이고, 후자는 비용과 실행 범위를 통제하는 장치다. 입력이 커졌다는 이유로 원문을 조용히 잘라 버리거나 새 세션으로 예산을 초기화하지 않도록 구분했다.

# 상태를 관리하는 엔진과 AI 실행을 분리했다

## 누가 무엇을 책임지는가

React·Vite 대시보드는 작업·계획·리뷰·예산을 보여 준다. Node.js·TypeScript 서버는 상태 전이와 다음 행동을 결정한다. SQLite에는 승인, 역할 배정, 단계 결과, 실행 기록을 저장하고, 원문과 결과 파일은 내용 해시로 식별한다.

| 구성               | 맡은 책임                          |
|------------------|--------------------------------|
| 대시보드와 중재 CLI     | 현재 상태 확인, 결정 전달, 작업 재개         |
| 엔진과 단계 관리        | 계획·승인·검토·예산·다음 단계 연결           |
| 역할 배정과 세션 연결     | 실행할 역할과 프로필 선택, 대화 재개 가능 여부 확인 |
| 공통 실행 런타임        | 실행 요청을 처리하고 세션·사용량·종료·오류를 반환   |
| Claude·Codex 어댑터 | 각 CLI의 옵션·권한·출력 형식을 변환         |
| Git·검사·자료 저장소    | 실제 코드, 검사 결과, 읽은 근거 확인         |

## 공통 실행이 모든 정책을 소유하지는 않는다

엔진과 호스트 리뷰는 같은 실행 계약을 쓰지만 각자의 승인과 예산을 소유한다. 런타임이 운영 DB를 대신 수정하거나 리뷰 승인을 만들어 내지 않는다. 이렇게 나누면 공급자 실행 방식을 고쳐도 작업 승인 규칙까지 바꾸지 않아도 된다.

핵심 코드: src/server/runtime/, src/server/turnRouting.ts, src/server/workGroupService.ts, src/server/engine/

# 역할을 특정 AI 회사에 고정하지 않았다

## 역할·프로필·배정·세션을 따로 기록한다

설계자·구현자·검토자라는 역할은 허용할 행동과 결과 형식을 설명한다. 실행 프로필은 공급자·모델·추론 강도·공급자별 옵션을 담는다. 배정은 어느 작업에서 어떤 프로필을 쓰는지와 그 버전을 연결한다. 세션은 해당 배정의 대화 기록이다.

이 구분이 없으면 중재자를 Claude에서 Codex로 바꿀 때 역할과 무관한 설정까지 다시 만들어야 한다. 개편에서는 배정과 기능 지원 여부를 먼저 확인한 뒤 공통 실행 요청으로 변환한다. 전역 배정을 작업별 배정으로 덮어쓸 수 있지만, 기록 없이 설정을 바꾸지는 않는다.

| 확인할 경계                 | 처리                      |
|------------------------|-------------------------|
| 공급자가 해당 역할과 옵션을 지원하는가  | 실행 전에 검사한다              |
| 다른 공급자에 같은 이름의 옵션이 있는가 | 이름만 보고 같은 의미라고 가정하지 않는다 |
| 모델이나 추론 강도를 지원하지 않는가   | 임의로 다른 설정으로 낮추지 않는다     |
| 이전 세션이 현재 배정과 호환되는가    | 공급자·참여자·배정 정보를 대조한다     |

## 현재 지원 범위

실제 어댑터는 Claude와 Codex 두 개다. 임의의 AI 도구를 연결하면 곧바로 동작하는 구조는 아니다. 새 공급자는 실행·권한·세션·사용량·결과 파싱 계약을 구현해야 한다. ultracode 같은 옵션도 공급자별 계약으로 남겨 공통 추론 강도와 혼동하지 않는다.

핵심 코드: src/shared/roles.ts, src/server/roleAssignments.ts, src/server/runtime/providers.ts

# 중재자가 바뀌어도 작업의 판단 맥락을 이어 간다

## 중재자는 상태를 읽고 다음 행동을 정한다

중재자는 어떤 작업이 진행 중인지, 무엇 때문에 멈췄는지, 남은 질문과 예산이 무엇인지 확인한다. 이미 승인된 범위 안의 수정과 다음 준비된 단계는 계속 진행한다. 제품 결정·인증·권한·예산처럼 실제로 부족한 조건은 이유를 남기고 요청한다.

대화의 마지막 문장만 보고 재개하지 않도록 현재 맥락 조회, 재개 상태 확인, 역할 배정 신원 확인 순서를 둔다. 중재자가 교체돼도 이전 승인과 질문이 사라지거나 이미 실행 중인 작업을 중복 시작하지 않게 하는 절차다.

## 자동 진행과 권한을 구분한다

자동 진행 정책은 매번 “계속할까요?”라고 묻는 비용을 줄인다. 그렇다고 구현자가 승인 범위를 넓히거나 리뷰 예산을 추가할 권한까지 갖는 것은 아니다. 로컬 완료, 공개 전달, 운영 설치 같은 행동은 해당 작업에 부여된 권한을 따라야 한다.

| 상태                 | 중재자가 할 일                         |
|--------------------|----------------------------------|
| 수정 가능한 검사 실패       | 원인과 영향 범위를 확인해 같은 작업에서 수정        |
| 실행 중인 작업           | 중복 명령이나 중복 리뷰를 보내지 않음            |
| 사용자 결정 또는 외부 자료 대기 | 질문과 필요한 근거를 구체화하고 독립 작업 가능 여부 확인 |
| 유효한 단계 완료          | 결과 묶음을 확인하고 다음 준비된 단계로 이동        |
| 예산 또는 인증 부족        | 상태를 보존하고 필요한 조치만 요청              |

핵심 계약: docs/mediation/policy.md, src/server/mediationAutonomy.ts

# 단계를 나누는 기준은 확인할 수 있는 결과다

## 파일마다 단계를 만들면 전달 비용이 커진다

설계·구현·테스트를 각각 별도 단계로 만들거나 파일별로 단계를 나누면 같은 내용을 여러 번 설명하고 검토하게 된다. 개편에서는 한 결과를 완성하는 작업을 단계 안의 체크리스트로 묶는다. “이 단계를 끝내면 무엇을 확인할 수 있는가?”를 먼저 적는다.

분리는 앞 단계의 결과가 있어야 판단할 수 있는 경우, 독립적인 검증이 필요한 경우, 실패 시 따로 되돌릴 필요가 있는 경우로 설명한다. 단계 수나 코드 줄 수를 일률적으로 제한하지 않는다.

| 예시                | 단계로 볼지 판단하는 기준                    |
|-------------------|-----------------------------------|
| 파서의 타입·구현·테스트     | 같은 입력과 출력 계약을 완성한다면 하나의 단계        |
| 파서 결과를 사용하는 검색 기능 | 앞 단계 결과와 별도로 사용자 결과를 검증할 수 있는지 확인 |
| 파일 이름 변경과 문서 정리   | 별도 완료 결과가 없다면 기존 단계의 체크리스트        |
| 외부 API 계약 결정      | 결정 후후에 가능한 작업이 달라지는지 확인           |

## 실행할 단계만 구체화한다

현재 단계에는 완료 조건, 검사 방법, 예산, 해결해야 할 질문이 필요하다. 미래 단계는 개요로 남길 수 있다. 기본적으로 한 단계를 실행하며, 외부 정보나 사용자 결정을 기다릴 때만 의존하지 않는 준비된 단계로 진행할 수 있다. 예산 소진을 기다림으로 바꿔 다른 실행을 시작하지 않는다.

핵심 코드: `src/shared/workGroups.ts`, `src/server/workGroups.ts`

# 계획을 바꿀 때 관련 없는 결과는 보존한다

## 큰 그림도 실제 결과에 맞춰 수정한다

미래 단계를 추가·분할·병합하거나 순서를 바꿀 수 있다. 다만 이미 실행과 연결되거나 완료된 단계의 구조를 덮어서 기존 결과의 의미를 바꾸지는 않는다. 진행 중인 단계의 전제가 바뀌면 변경 기록과 재계획을 연결한다.

어떤 단계가 영향을 받는지 계산해 관련 계획만 다시 확인한다. 무관한 승인·세션·검사 결과는 유지한다. 모든 단계가 처음부터 다시 시작하면 점진적으로 작업하는 이점이 사라지기 때문이다.

## 실패한 재계획을 완료로 취급하지 않는다

재계획 중 오류가 나면 해당 변경을 대기 상태로 보존한다. 이전 계획으로 실행하던 응답이 늦게 도착해도 새 계획의 결과로 받아들이지 않는다. 같은 변경을 다시 처리할 때는 기존 기록과 예산을 이어 사용한다.

| 다음 단계에 넘기는 것     | 이유                             |
|------------------|--------------------------------|
| 검증된 로컬 커밋        | 구현의 실제 출발점을 고정                 |
| 승인된 계획과 검사 근거    | 어떤 계약을 만족했는지 확인                |
| 실제로 사용한 위키 버전    | 판단 근거가 바뀌었는지 확인                |
| 남은 질문과 이관한 지적    | 누락되거나 다시 발견되는 일을 줄임            |
| 적용 범위가 있는 사용자 결정 | 한 단계의 결정을 전체 작업으로 과도하게 확대하지 않음 |

핵심 코드: `src/server/revisionLedger.ts`, `src/server/workGroupService.ts`,  
`src/server/workGroupAdapter.ts`

# 큰 원문은 위치를 기억하며 나눠 읽는다

## 첫 입력에는 자료의 위치와 현재 판단을 담는다

계획 시작 시 목표, 진행 기록, 자료 목록과 참조를 전달한다. AI는 서버가 관리하는 읽기 도구로 파일·Git 변경·저장 자료를 조회한다. 전체 폴더와 대화 이력을 한꺼번에 넣는 대신 현재 질문을 해결하는 데 필요한 원문을 요청한다.

읽은 조각에는 출처, 버전, 내용 해시, 바이트 범위, 다음 위치가 연결된다. 같은 문서의 앞부분을 반복해서 보내거나 버전이 다른 조각을 하나의 원문처럼 합치지 않도록 확인한다. 실제로 전달된 조각만 근거로 인용할 수 있다.

| 입력 제어           | 의미                         |
|-----------------|----------------------------|
| 조각·묶음의 크기 제한    | 한 호출에 몰아넣지 않고 이어 읽기 위한 제한  |
| 필수 자료의 읽기 완료 확인 | 결정을 좌우하는 원문을 끝까지 읽었는지 확인   |
| 추가 읽기가 남은 완료 응답 | 최종 계획으로 승인하지 않고 중간 응답으로 처리 |
| 두 번 연속 진전 없음    | 새 근거나 질문 해결 없이 반복하는 흐름 중단  |

## 큰 입력과 컨텍스트 손실은 다른 실패다

누적 입력 크기만으로 강제 요약이나 새 세션을 만들지 않는다. 실제 컨텍스트 한도나 세션 손실이 확인됐을 때만 제한된 체크포인트 복구를 사용한다. 공급자·모델·설정과 누적 예산을 유지한다. 자료를 읽은 기록이 사라졌다면 재조회 이유를 남긴다.

핵심 계약: docs/guarded-planning.md

# 위키는 필요한 근거를 다시 찾는 통로다

## 색인을 본 것과 본문을 읽은 것을 구분한다

위키 색인에는 문서의 위치와 관련 주제가 있다. 처음에는 관련 문서 일부만 선택하고, 더 필요한 자료를 질의해 읽는다. 모든 위키를 프롬프트에 붙이거나 색인에 나타난 모든 문서를 판단 근거로 등록하지 않는다.

실제로 조회한 문서의 경로·버전·내용을 기록한다. 읽지 않은 문서가 바뀌었다는 이유로 현재 작업을 모두 무효화하지 않는다. 반대로 읽은 원문이 바뀌면 관련 판단을 다시 확인해야 한다.

## 외부 자료의 변경은 결론을 자동으로 바꾸지 않는다

위키 본문의 해시와 외부 원문의 해시는 다른 정보를 가리킨다. 외부 자료가 갱신되면 “확인이 필요하다”는 상태를 남긴다. 새 내용이 질문인지 결정인지, 어느 플랫폼에 적용되는지, 기존 근거와 충돌하는지 판단한 뒤 결론을 고쳐야 한다.

| 구분      | 확인할 것                        |
|---------|------------------------------|
| 검색 정확도  | 필요한 자료를 찾았는가, 불필요한 자료를 제외했는가 |
| 근거의 최신성 | 실제로 읽은 버전이 현재 원문과 맞는가        |
| 전달량     | 얼마나 많은 본문 바이트를 모델에 보냈는가      |
| 실제 비용   | 공급자가 보고한 입력·캐시·출력 사용량은 얼마인가  |

검색 정확도가 좋아졌다고 토큰이 줄었다고 말할 수는 없다. 더 관련 있는 문서를 많이 선택하면 전달량이 늘 수도 있다. 위키 수정 제안도 경로·상태·내용 해시와 역할 권한을 확인한 뒤 반영하며, 폴더 전체를 임의로 고칠 권한을 주지 않는다.

핵심 계약: docs/memory-retrieval.md, src/server/wikiEvidence.ts

# 외부 자료 갱신은 조회와 모델 전달을 나눈다

## 등록한 출처만 주기적으로 확인한다

Slack·Jira·Figma 링크를 등록하면 REST 수집기가 기본 15분 간격으로 조회 시점을 확인한다. 시작할 때 밀린 조회도 확인한다. 닫힌 작업에서만 사용하는 출처는 정기 조회 대상에서 빠진다. 외부 자료가 없는 작업에는 조회할 대상이 없다.

수집 자체는 LLM을 호출하지 않는다. 변경분은 중재자가 시작·재개할 때 받는다. 커넥터 방식은 해당 호스트의 자료 수집이 필요하며, REST 인증과 커넥터 인증을 자동으로 같은 것으로 취급하지 않는다.

| 출처    | 변경 감지                      | 비용이 남는 부분              |
|-------|----------------------------|------------------------|
| Jira  | 수정 시각 확인 후 본문 수집, 댓글 별도 확인 | 메타데이터와 댓글 조회           |
| Slack | 등록한 대화의 페이지를 재확인           | 과거 수정·삭제 감지를 위한 API 읽기 |
| Figma | 파일 버전과 관련 노드 내용 비교         | 버전 조회, 바뀐 노드와 댓글 확인    |

## 계획에는 Figma 링크와 기능 흐름만 둔다

계획 단계는 화면별 링크를 디자인의 위치로 보관한다. 레이아웃·크기·간격·폰트·색상·노드 트리·스크린샷을 모아 계획에 옮기지 않는다. 구현자가 해당 화면을 만들 때 필요한 자료를 직접 확인한다. 링크나 캐시된 이미지가 있다는 사실만으로 디자인을 검증했다고 판단하지 않는다.

자료 변경이나 장기간 미확인은 실행·결과 수락 전 재검토가 필요하다. 접근 거부와 요청 제한을 최신 상태의 근거로 쓰지 않는다. 같은 본문을 다시 전달하지 않아도 API 조회나 공급자의 캐시 입력 비용은 남을 수 있다.

핵심 계약: docs/external-evidence.md

# 계획의 대화를 구현까지 이어 쓴다

## 같은 판단을 다시 설명하는 비용을 줄인다

설계자가 이미 코드와 요구사항을 이해했는데 구현을 새 세션에서 시작하면 판단 근거를 다시 요약하고 전달해야 한다. 개편에서는 계획과 구현의 배정이 호환될 때 같은 대화를 이어 사용한다. 원문 전체를 다시 붙이는 별도 전달 문서를 필수로 만들지 않는다.

이 연결은 세션 이름만으로 결정하지 않는다. 공급자, 참여자, 역할 배정과 실행 조건을 확인한다. 호환되지 않는 세션을 억지로 재개하거나 임의의 다른 모델로 대체하지 않는다.

## 검토자의 판단 맥락은 별도로 유지한다

검토자는 고정된 계획과 구현 결과를 독립적으로 본다. 첫 리뷰는 전체 변경을 확인하고, 수정 후에는 같은 리뷰 맥락에서 바뀐 부분과 미해결 지점을 확인한다. 범위를 넓혀야 한다면 새 반증이나 공통 계약의 영향처럼 이유가 있어야 한다.

| 재사용 대상            | 재사용 조건                        |
|-------------------|-------------------------------|
| 계획에서 구현으로 이어지는 대화 | 현재 배정과 세션이 호환됨                |
| 수정 후 리뷰 대화        | 같은 논리 작업과 누적 예산을 유지함          |
| 읽은 외부 자료          | 같은 작업·역할·세션의 전달 기록과 버전 확인     |
| 검사 결과             | 입력·도구·실행 환경·계획·로그가 재사용 조건을 충족 |

실패·취소된 전달은 읽음으로 기록하지 않는다. 새 세션에는 필요한 자료를 다시 보낼 수 있다. 이 구조는 반복 입력을 줄일 수 있는 조건을 마련하지만, 실제 토큰 절감률은 같은 조건의 비교 실행으로 따로 측정해야 한다.

# 승인과 리뷰에서 내용이 바뀌지 않았는지 확인한다

## 계획 해시는 동일성을 확인하는 장치다

설계자와 검토자가 서로 다른 계획을 보고 동의하는 일을 막기 위해 정규화된 계획의 SHA-256을 사용한다. 현재 승인도 그 계획에 연결한다. 해시가 같다는 사실은 같은 내용을 가리킨다는 뜻이며, 계획이 옳다는 증명은 아니다. 타당성은 별도 검토가 맡는다.

리뷰에는 지적 ID와 처리 상태를 남긴다. 수정·수용·반박·범위 밖 이관을 구분하고, 응답에서 지적이 빠졌다고 해결된 것으로 처리하지 않는다. 불완전한 리뷰는 이전 지적을 지우거나 변경분 기준을 전진시키지 못한다.

## 리뷰를 반복해도 검토한 코드가 분명해야 한다

기준 커밋, 후보 커밋, 파일 목록과 변경 내용을 고정한 뒤 리뷰한다. 수정 중인 작업 폴더를 동시에 검토하면 리뷰가 끝났을 때 실제 코드가 달라질 수 있다. 후속 리뷰는 바뀐 지적과 영향을 받는 호출부를 확인하며, 매번 전체 작업을 처음부터 재감사하지 않는다.

| 확인 대상 | 통과 조건                   |
|-------|-------------------------|
| 계획 승인 | 현재 계획의 내용과 승인 대상이 일치    |
| 리뷰 지적 | 기존 ID의 처리 결과와 근거가 남아 있음 |
| 검사 결과 | 검토한 코드에 대한 실행 결과임       |
| 로컬 완료 | 검토한 트리와 실제 커밋의 트리가 일치   |

범위 밖 개선은 다음 작업에 넘길 근거로 보존한다. 작은 개선을 계속 현재 작업에 포함해 리뷰가 끝나지 않는 흐름을 피하기 위한 선택이다.

# 권한과 프로세스 종료를 결과 판정에 포함한다

## 역할에 필요한 실행 권한만 준다

계획·리뷰는 읽기, 구현은 쓰기, 단순 확인 응답은 도구 권한 없이 실행하는 계약을 사용한다. 파일 수정 가능 여부를 특정 공급자 이름에 묶지 않는다. 해당 역할과 실행 도구가 요청한 권한을 지원하는지 먼저 검사한다.

모델이 “완료”라고 출력했더라도 프로세스가 정상적으로 끝났는지와 결과 형식이 유효한지를 확인해야 한다. 출력 일부만 읽고 성공을 확정하면 뒤늦게 나온 오류나 잘린 응답을 놓칠 수 있다.

## 최소한 작업의 늦은 응답을 막는다

취소·재계획·재시작이 일어날 때 실행 신원과 상태를 대조한다. 이전 실행이 늦게 끝나도 최신 계획의 결과나 승인 상태를 덮어쓸 수 없다. 프로세스를 정리할 때도 PID만 보지 않고 시작 신원을 확인해 다른 프로세스를 잘못 종료하지 않도록 한다.

| 실패 장면                 | 처리 원칙                          |
|-----------------------|--------------------------------|
| 취소 후 성공 응답 도착         | 취소된 실행의 결과로 남기고 현재 상태에 반영하지 않음 |
| 같은 포트에 기존 서버가 있음      | 서버 소유권을 확인하기 전에 실행을 복구하지 않음    |
| 프로세스는 끝났지만 JSON이 불완전함 | 정상 완료로 처리하지 않음                 |
| 중간에 사용량 보고가 없음        | 0으로 채우지 않고 미제공으로 기록            |

핵심 코드: `src/server/processRunner.ts`, `processSupervisor.ts`, `runtime/service.ts`

# 파일과 화면의 진행 표시도 원본과 맞춰 확인한다

## 편의용 경로보다 내용 해시를 신뢰한다

계획·리뷰·구현 결과를 파일로 남기면 대화가 길어져도 필요한 원문을 다시 읽을 수 있다. 다만 별칭 파일이 다른 내용으로 교체될 수 있으므로 내용 해시로 고정된 자료를 기준으로 삼는다. DB의 참조와 실제 파일을 읽을 때 해시를 대조한다.

화면은 서버의 실행 기록과 이벤트를 받아 진행 상태를 보여 준다. 스트리밍 문장이 화면에 보인다는 사실만으로 실행 완료나 승인이 성립하지 않는다. 연결이 끊겼다가 돌아와도 저장된 상태를 기준으로 확인한다.

## 변경 범위 검사는 코드의 의미 검증과 다르다

승인한 경로와 변경 규칙을 벗어났는지 확인할 수 있다. Swift 주석·문자열을 구분해야 하는 검사에서는 단순 문자열 치환으로 판단하면 오탐과 누락이 생긴다. 범위 규칙을 적용할 때도 해당 언어의 표현을 고려해야 한다.

| 장치         | 확인하는 것           | 확인하지 못하는 것        |
|------------|------------------|-------------------|
| 내용 해시      | 읽은 자료가 같은 바이트인지  | 요구사항이 올바른지        |
| 이벤트와 저장 상태 | 실행의 진행·종료 기록     | 사용자가 기대한 기능이 맞는지  |
| 경로·변경 규칙   | 승인 범위 안의 수정인지    | 앱 동작의 정확성         |
| 테스트·리뷰     | 해당 경계의 계약과 변경 위험 | 실행하지 않은 UI와 외부 환경 |

서로 다른 증거를 하나의 성공 표시로 합치지 않는 것이 이 설계의 중요한 제한이다.

## 단계 완료와 원격 전달을 별도로 판정한다

### 다음 단계는 검증된 로컬 커밋에서 시작한다

단계가 끝나려면 승인된 계획과 리뷰 결과가 실제 로컬 커밋에 연결돼야 한다. 다음 단계는 그 커밋과 결과 묶음을 받는다. 모델이 보고한 커밋 문자열만 믿지 않고 기준 커밋과의 관계, 트리, 작업 상태를 확인한다.

여러 결과가 서로 다른 커밋에서 갈라졌다면 엔진이 커밋 전 통합 상태를 준비한다. 러너가 충돌을 해결하고 검토를 통과한 뒤 부모 커밋을 보존한 통합 커밋을 만든다. 변경이 없는 통합도 검토한 트리가 기준과 같은 지 확인해야 한다.

| 단계       | 확인할 사실                   |
|----------|--------------------------|
| 개별 단계 완료 | 검토한 내용과 로컬 커밋이 일치        |
| 최종 통합    | 모든 단계 결과를 포함하는 커밋 관계가 성립 |
| 푸시 실행    | 승인한 대상과 범위로 전달           |
| 전달 완료    | 원격 OID에서 실제 반영 확인        |

### 푸시 응답이 없어도 무조건 다시 보내지 않는다

네트워크 오류로 결과를 받지 못해도 원격에는 이미 반영됐을 수 있다. 먼저 원격 커밋을 확인하고 저장된 실행 기록과 대조한다. 이미 반영된 작업을 중복 실행하지 않고 전달 상태를 복구한다.

파일럿에서는 격리된 bare 원격으로 이 흐름을 확인했다. 이 결과가 임의의 실제 서비스 배포와 운영 데이터 변경까지 검증했다는 뜻은 아니다.

# 실행 횟수와 비용은 작업 전체에서 누적한다

## 읽기 호출 수와 논리적 리뷰 횟수를 구분한다

리뷰가 원문을 여러 번 나눠 읽는다고 매번 별개의 리뷰를 시작한 것은 아니다. 읽기와 최종 판단을 하나의 논리적 리뷰로 연결할 수 있어야 큰 자료도 정상적으로 검토할 수 있다. 동시에 실제 모델 호출마다 발생한 사용량과 시간은 빠짐없이 기록해야 한다.

수정과 확인 응답이 추가되는 경우에는 해당 정책에 따라 리뷰 예산을 사용한다. 실패한 실행을 성공처럼 지우거나, 새 커밋·새 세션·서버 재시작으로 같은 작업의 사용량을 초기화하지 않는다.

## 한도를 넘으면 기록을 유지한 채 멈춘다

리뷰 횟수, 모델 사용량, 실행 시간은 서로 다른 제한이다. 사용량 제한이 확인되면 이유와 재시도 시점을 남긴다. 재시도는 취소할 수 있어야 하고, 오래된 실행의 재시도가 새 계획을 덮어쓰면 안 된다.

| 기록          | 해석                          |
|-------------|-----------------------------|
| 실제 모델 실행 수  | 실행 도구가 실제로 시작된 횟수           |
| 입력과 캐시 입력   | 공급자의 보고 의미에 맞게 구분, 중복 합산 금지 |
| 출력 토큰       | 응답 생성 사용량                   |
| 호출별 실행시간 합계 | 전체 작업의 경과 시간과 다름            |
| 값이 없는 사용량   | 미제공이며 0이 아님                 |

파일럿에서도 추가 예산과 리뷰 횟수를 명시적으로 부여했다. 이는 제한이 사라졌다는 뜻이 아니라 같은 작업의 기록 위에 추가 권한을 연결했다는 뜻이다.

# 검사 재사용의 범위를 좁혀 검증했다

## 같은 코드라는 이유만으로 모든 검사를 생략하지 않는다

현재 재사용 경로는 알려진 Swift 동시성 정책 정적 검사 한 종류를 대상으로 한다. 관련 입력, 검사 도구, 실행 환경, 승인 계획과 결과 로그를 해시로 확인한다. Git 추적 대상이 아닌 관련 Swift 파일도 입력 집합에 포함해야 한다.

검사 결과를 저장하는 도중 중단됐다면 다음 실행에서 완료 여부와 근거를 복구한다. 비교할 근거가 없다는 상태를 “변경 없음”으로 처리하지 않는다. CLI가 제출한 실행 보고서와 독립적으로 확인한 실행 증거의 차이도 남는다.

## 실제 앱에서 겪은 실패가 확인 범위를 바꿨다

Swift 6 전환 과정에서는 빌드 로그 일부나 재사용된 산출물만 보고 성공을 판단할 위험이 드러났다. WebKit 종료 과정의 오류처럼 컴파일 뒤 실행에서 나타나는 문제도 있었다. 운영체제 로그와 앱 프로세스 오류를 구분하지 않으면 잘못된 실패를 추적할 수 있었다.

| 필요한 결과      | 별도로 필요한 확인                  |
|-------------|-----------------------------|
| 정적 정책 위반 없음 | 현재 입력으로 해당 검사 실행 또는 유효한 재사용 |
| 컴파일 성공      | 실제 빌드 종료 결과                 |
| 앱 실행 가능     | 대상 시뮬레이터 설치·실행과 앱 오류 확인     |
| 화면과 제스처 일치  | UI 실행·화면·상호작용 증거            |

따라서 정적 검사 캐시를 전체 단위 테스트·빌드·UI 검증의 대체물로 확장하지 않았다. 효율보다 검증 결과가 의미하는 범위를 먼저 고정한 선택이다.

# 엔진을 고치지 않고 끝내는 실제 파일럿을 수행했다

## 첫 실패를 보존하고 수정 후 다시 시작했다

첫 파일럿은 예산 처리와 계획 확정 경로의 결함으로 중단됐다. 중단 기록을 유지한 채 관련 계약을 수정하고 검토했다. 그 실행을 이어 붙여 성공으로 만들지 않고 새 파일럿에서 전체 흐름을 다시 확인했다.

두 번째 파일럿은 실제 Claude·Codex CLI로 수행했다. 진행 중 엔진 파일을 바꾸지 않았고, 비교 대상 243개 파일의 해시가 유지됐다. 마지막에는 모든 단계를 닫고 최종 통합 결과를 격리된 bare 원격에 전달했다.

| 파일럿에서 확인한 흐름   | 결과                                |
|----------------|-----------------------------------|
| 큰 그림에서 첫 단계 실행 | 미래 단계는 개요로 두고 현재 단계만 상세화          |
| 중재자 교체         | Claude에서 Codex로 전환하며 배경과 진행 기록 확인 |
| 미래 단계 분할       | 영향 없는 결과와 승인 보존                   |
| 큰 원문 읽기        | 조각으로 나눠 읽고 필요한 근거 연결              |
| 서버 재시작         | 파일럿 DB 60개 테이블의 상태 보존             |
| 나머지 단계와 최종 통합  | 5개 단계 완료, 원격 반영 확인                |

최종 대상 프로젝트의 검사 77개가 통과했고, 고의로 필요한 export를 제거한 반대 검사에서는 실패했다. 통과 숫자만으로 판단하지 않고 검사가 실제 결함을 거부하는지도 확인했다.

# 파일럿의 사용량과 검증 범위를 함께 기록했다

## 실제 실행에서 관측한 수치

| 항목              | 관측 결과                 |
|-----------------|-----------------------|
| 모델 실행           | 48회                   |
| 입력 토큰           | 22,602,633            |
| 위 입력에 포함된 캐시 입력 | 18,134,381            |
| 출력 토큰           | 620,113               |
| 모델 실행시간 합계      | 6,566초                |
| 완료와 전달          | 5개 단계 완료, 격리 원격 반영 확인 |

캐시 입력은 전체 입력에 포함된 값이며 다시 더하지 않는다. 실행시간은 호출별 시간의 합계로, 작업 시작부터 완료까지의 경과 시간과 다르다. 이전 엔진과 같은 작업·모델·설정으로 비교한 실행이 없으므로 이 숫자를 절감률로 표현하지 않는다.

## 공개 코드 검사와 실제 파일럿은 다른 증거다

문서 갱신 전 기준에서 개발본 검사 1,719개, 호스트 리뷰 Python 검사 160개, 공개본 검사 1,381개 통과를 확인했다. 개발본에는 예상 실패 2개와 제외 7개, 공개본에는 제외 3개가 별도로 있다. TypeScript 검사와 웹 빌드도 통과했다. 이 수치는 검사 범위를 설명하며 외부 서비스 전체의 정상 동작을 보장하지 않는다.

파일럿의 외부 자료는 저장된 시험용 스냅샷이었다. 실제 Slack·Jira·Figma를 모두 연결한 종단 간 검증은 포함하지 않는다. 세션별 읽음 기록은 확인했지만 실제 재전송 전체를 계측하지 않아 “중복 전송 0회”라고 결론 내릴 수 없다. 예산과 리뷰 횟수 추가도 있었으므로 완전 무인 실행으로 소개하지 않는다.

# 다음 검증은 실제 사용 조건에서 비교한다

## 이번 개편으로 확보한 것

현재 구현은 큰 그림을 수정하면서 단계를 이어 가고, 원문을 필요할 때 나눠 읽고, 역할과 실행 도구를 분리한다. 계획에서 구현으로 대화를 이어 쓰며, 승인·리뷰·검사·커밋·원격 전달을 각각 확인한다. 실제 파일럿에서는 엔진을 중간 수정하지 않고 이 흐름을 끝냈다.

운영 반영에서는 기존 작업 데이터와 설정을 보존한 설치·재시작을 확인했다. 설치 검증과 파일럿은 서로 다른 증거다. 같은 실행 파일을 다시 활성화한 복구 검사가 서로 다른 구버전·신버전 바이너리 사이의 복구까지 증명하지는 않는다.

## 아직 검증을 넓혀야 하는 부분

| 다음 확인              | 필요한 이유                           |
|--------------------|----------------------------------|
| 실제 외부 서비스 연결 실행    | 저장 스냅샷 검사와 인증·요청 제한·실시간 변경을 구분   |
| 물리적인 재전송 계측        | 읽음 기록만으로 중복 호출 전체를 알 수 없음        |
| 같은 작업의 변경 전후 비교    | 토큰·시간·비용 절감률을 판단할 비교군 필요         |
| 긴 구현 메모와 조회 위치 안내  | 메모 길이 제한과 원문 위치 안내가 판단을 방해하는지 확인 |
| 검사 증거 전달과 예산 사전 안내 | 불필요한 추가 리뷰와 뒤늦은 예산 중단을 줄일 여지     |

이 작업에서 얻은 기준은 AI가 오래 실행됐다는 사실보다 무엇을 읽고, 무엇을 승인받아, 어떤 코드를 검증해 전달했는지 설명할 수 있어야 한다는 것이다. 기능을 추가할 때도 이 연결을 유지하면서 필요한 검증만 확장한다.

공개 코드: <https://github.com/ChanSeul/consensus-room>

관련 문서: docs/guarded-planning.md · docs/external-evidence.md · docs/memory-retrieval.md · docs/usage-report.md